

REST API Design

IF3110 Web-Based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Why design an API?

Suppose GET /products returns:

- An **array of products** when products exist.
- The string "No products" when there are no products.

This creates problems because:

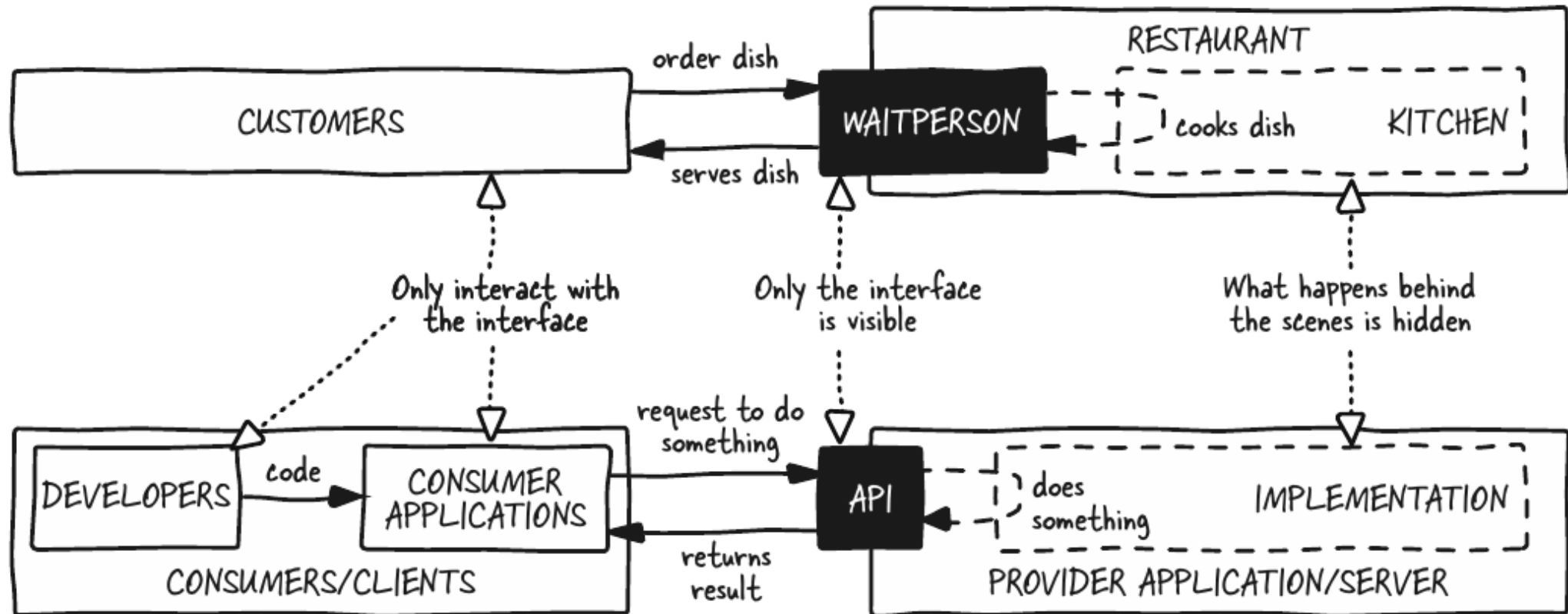
- Clients must handle **two different response types**.
- Clients cannot rely on a **single documented schema**.
- Client-side code becomes more complex and error-prone.

A designed contract gives each operation a stable input, output, error meaning, and migration path. For this example, an empty collection is still
`{ "items": [] }.`

Why API design matters

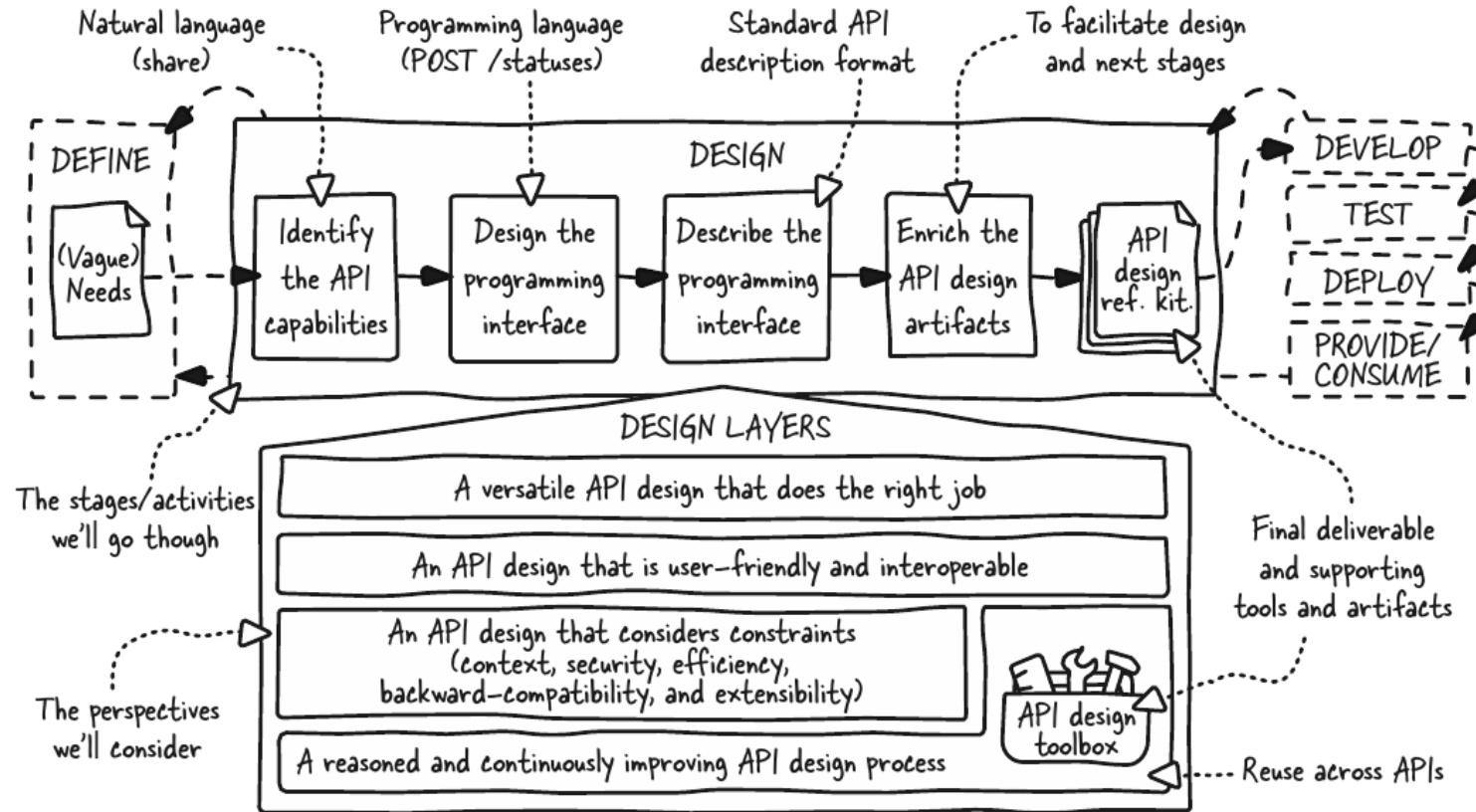
Benefit	Design mechanism	Practical effect
Developer productivity	Clear names and consistent behavior	Less guesswork and integration rework
Architectural flexibility	A stable business-facing contract	Internal changes affect fewer clients
Efficient operation	Bounded data and appropriate caching	Less avoidable traffic and processing
Safer evolution	Compatibility review and migration	Lower risk for existing consumers

A stable interface hides internal complexity



The backend can change while consumers continue to use the same compatible contract.

Layered design approach, Lauret Arnaud



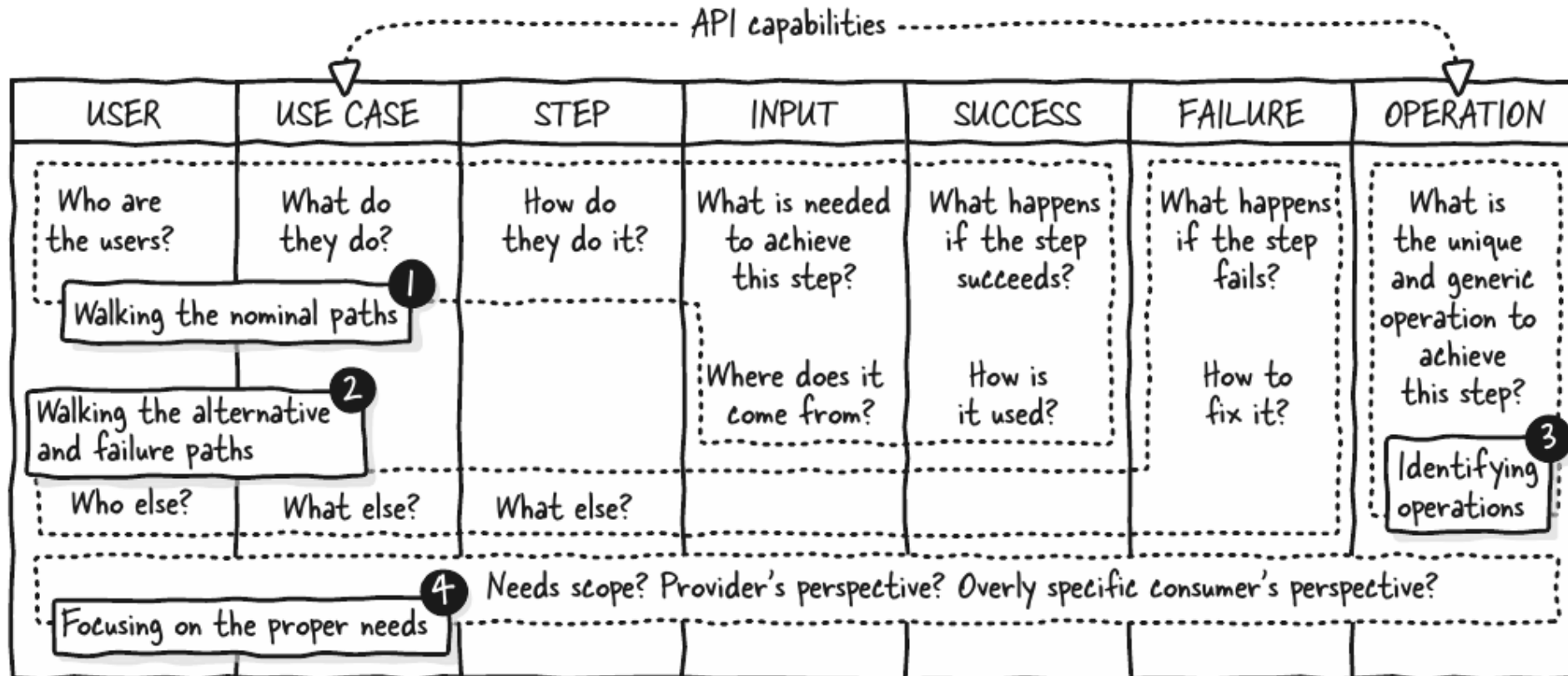
Four layers: the right job, usability, real-world constraints, and continuous improvement.

REST API Design

Layer 1 :

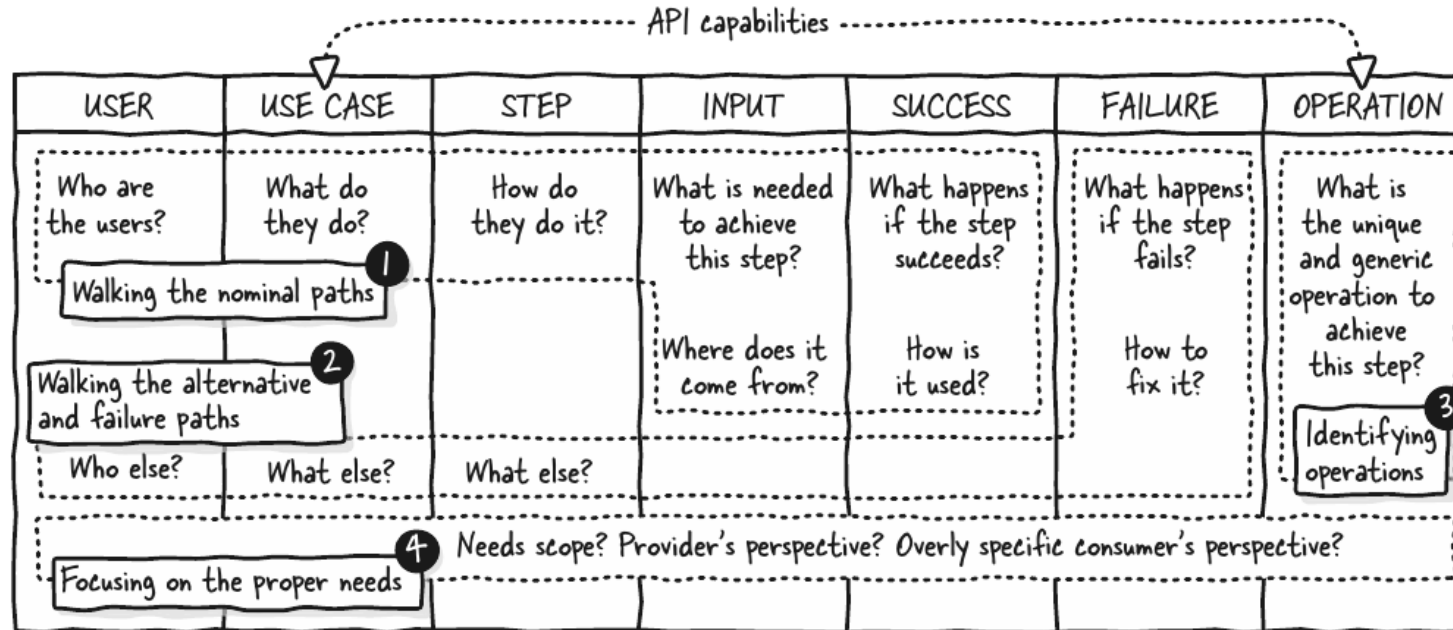
Does the right Job, Understand user needs and design the capabilities, resources, and operations that satisfy them

Layer 1: capabilities before HTTP paths



Library example: a member searches for a book, borrows it, and receives a loan confirmation.

User needs and the Capabilities Canvas



Lauret, 2nd ed., Figure 2.3 (p. 29)

Capabilities Canvas can help understand user needs :

1. Uncover what consumers need to achieve (walking nominal path or the “happy path”)
2. First walk the successful use case then see the alternatives and failure
3. Trace where inputs came, and how consumer use outputs.
4. consolidate repeated steps into versatile operations

Capabilities Canvas: borrowing a library book

User	Use case	Step	Input	Success	Failure / recovery	Operation
Member	Borrow a book	Find a book	Search terms from member	Matching books Choose one	No matches Change search terms	Search books
Member	Borrow a book	Confirm borrowing	Book ID from selection Member ID from app	Loan ID and due date Show confirmation	Unavailable book Choose another Missing member ID Supply it	Create a loan

Alternative: a known book ID lets the member skip search and request borrowing directly.

Scope: borrowing books. Returning books and paying fines are separate use cases.

Layer 1: resources and HTTP semantics

Intent	Example	Contract outcome
Read a book	GET /books/b1	200 with book data, or 404
Borrow a book	POST /loans	201 with loan and Location
Replace / modify	PUT /books/b1 or PATCH /books/b1	Define writable state and patch format
Remove a resource	DELETE /reservations/r1	204 when no response content
Explain a failure	Invalid input / unavailable book	400 for input, 409 for this conflict

REST API Design

Layer 2 :

Is user friendly and interoperable. Make the API easy to understand and use through consistent naming, standard formats, useful errors, and predictable behavior.

Layer 2: predictable, usable data

Clear vocabulary

Use meaningful names and a consistent convention such as camelCase.

Unambiguous formats

Use RFC 3339 timestamps and explicit currency and country codes.

Data ready for use

Separate values into fields and document units, precision and nullability.

```
{
  "status": "active",
  "dueDate": "2026-10-17",
  "createdAt":
    "2026-09-27T09:00:00Z",
  "fine": {
    "amountMinor": 50000,
    "currency": "IDR"
  },
  "countryCode": "ID"
}

// Document money units and scale
```

Layer 2: bounded collections and useful filters

A bounded response

Set a default and maximum page size. Return a way to request the next page.

An intentional paging model

Choose offset or cursor pagination based on ordering, change rate and access patterns.

Explicit filtering behavior

Define supported filters, combinations, invalid values and whether each is required.

A predictable empty result

Return an empty collection when a valid search finds no matching books.

Layer 2: actionable error responses

An honest HTTP outcome

Use a status that reflects the failed operation instead of disguising failure as success.

A structured problem

RFC 9457 defines application/problem+json and common problem fields.

Useful recovery information

Identify the problem safely. Define any field-level error extensions in the contract.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
```

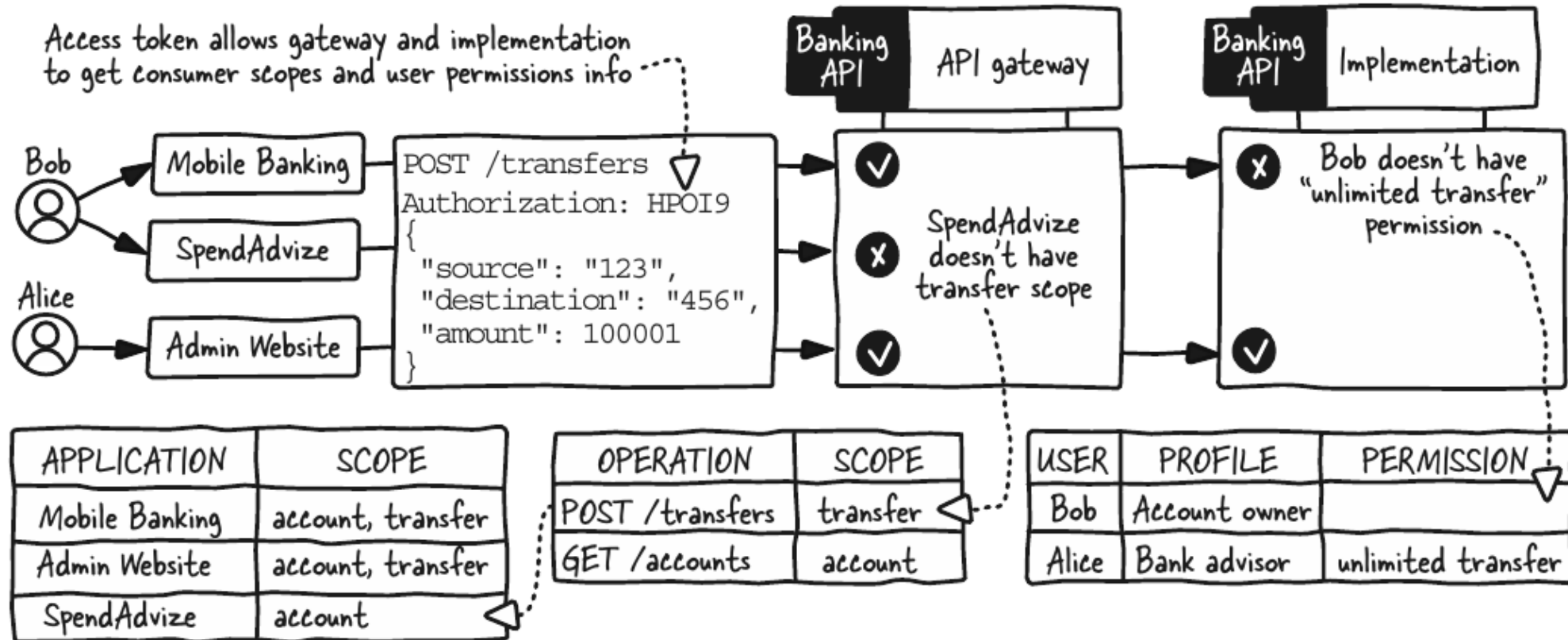
```
{
  "type": "urn:problem:invalid-input",
  "title": "Invalid input",
  "status": 400,
  "detail": "memberId is required"
}
```

REST API Design

Layer 3 :

Consider real world constraint. Address Security, efficiency, technical context, backward capability, and extensibility.

Layer 3: scopes and user permissions



A scope limits an application's access. User and object permissions still need enforcement.

Layer 3: preventing BOLA (Broken Object-level Authorization)

The vulnerability

Broken Object-Level Authorization lets a caller access an object without permission.

The required decision

Check the user, requested action and particular loan on every relevant operation.

A testable access rule

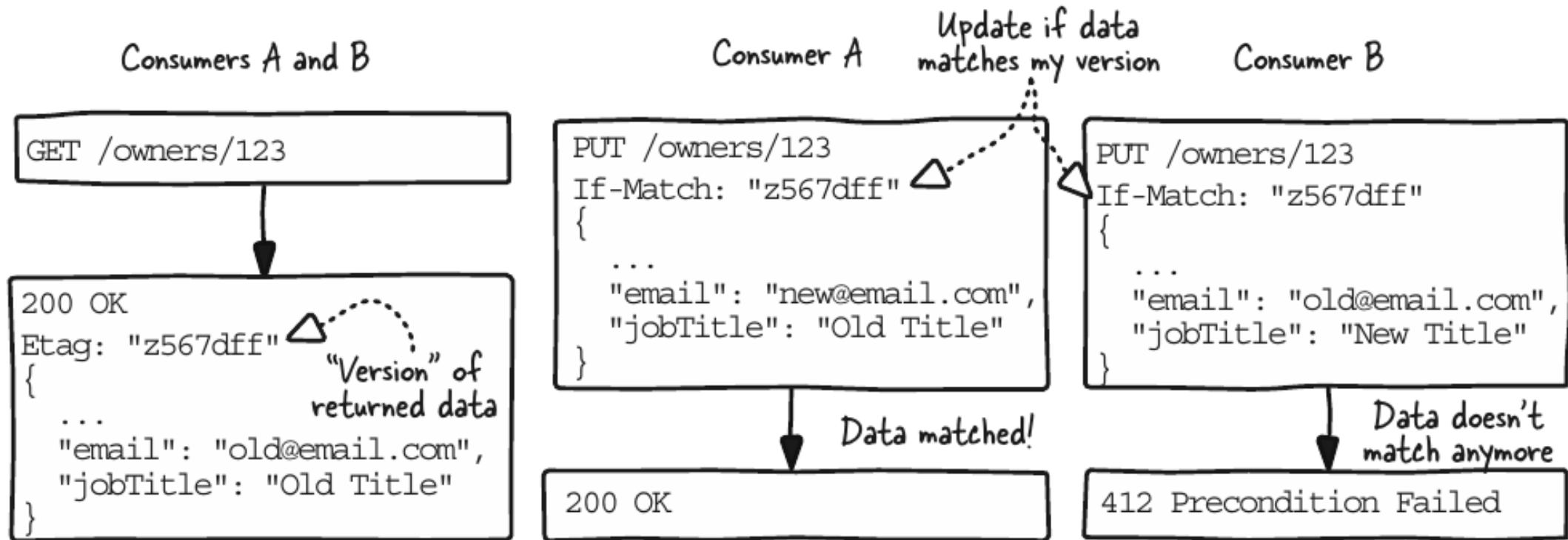
Alice may read her own loan. Changing its ID must not expose Bob's loan.

```
# Alice owns loan 11
GET /loans/11
# Allowed by the access policy

# Bob owns loan 12
GET /loans/12
# Alice must not receive Bob's loan

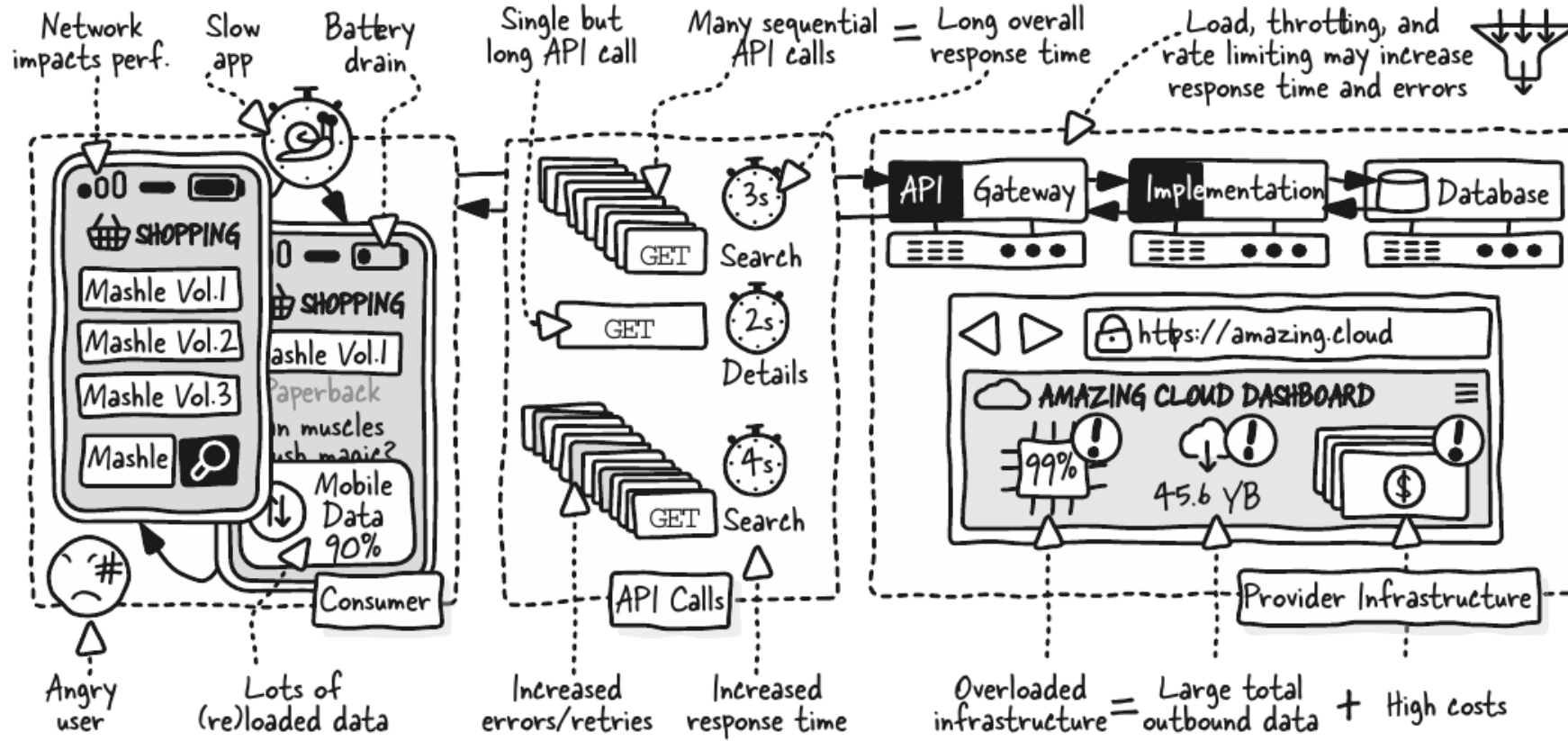
# Example concealment policy:
HTTP/1.1 404 Not Found
```

Layer 3: protecting updates with conditions



If-Match prevents an update from silently overwriting a newer representation.

Layer 3: the cost of an inefficient API



Excessive data and repeated calls can increase client latency, infrastructure load and outbound traffic.

Layer 3: efficiency controls and their limits

Control	Example	Important limit
Freshness	Cache-Control: max-age=60	Only reuse when cache policy permits
Revalidation	If-None-Match with a stored ETag	304 transfers no representation body
Rate limits	429 and a documented retry policy	Limit costly work, not only call count
Bulk operations	Create several records per request	Bound batch size and define outcomes

Layer 3: compatible evolution

Change	Possible consumer impact	Safer approach
Rename sts to status	Existing code cannot find sts	Plan a documented migration
Require a new input	Old requests become invalid	Preserve old behavior or version deliberately
Add an enum value	Exhaustive logic may fail	Define extensibility and test consumers
Refactor storage	No required API change	Preserve meanings and behavior

REST API Design

Layer 4 :

Uses a reasoned, continuously improving design process. Record decisions, establish guidelines, review designs, automate checks, and incorporate feedback.

Layer 4: a reviewed OpenAPI contract

Specification-first design

Describe operations and schemas early so consumers can review the planned interface.

Shared design evidence

Use the contract for documentation, examples, mocks and appropriate tooling.

Agreement with the server

Review changes and check behavior. A document alone cannot prevent contract drift.

```
# OpenAPI 3.1 fragment
paths:
  /loans:
    post:
      operationId: createLoan
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref:
                '#/components/schemas/LoanCreate'
      responses:
        '201':
          description: Loan created
```

Layer 4: guidelines and automated checks

Shared conventions

Agree on naming, errors and pagination, with reasons and examples.

Spectral in CI

Lint the API description for selected rules before accepting a change.

Human review still matters

Linting cannot prove correct user needs, authorization or runtime behavior.

```
rules:
  operation-id-required:
    description: Add an operationId
    severity: error
    given: $.paths.*[get,post]
    then:
      field: operationId
      function: truthy
```

```
# Example command in CI:
# spectral lint openapi.yaml
```

Poor design creates downstream costs

Design weakness	Likely consequence	Preventive practice
Internal details in the contract	Clients depend on backend structure	Business-facing resources
Missing object permission check	Unauthorized disclosure or modification	Object-level authorization
Unbounded collection results	High transfer cost and memory pressure	Pagination and query limits
Unplanned breaking change	Existing client behavior fails	Compatibility review
Vague or misleading errors	Slow debugging and frustrated integrators	Actionable error contracts

REST API Design

Go in depth with REST API Design Principle

REST API

REST stands for *Representational State Transfer*. Roy Fielding introduced it as an architectural style for networked systems.

- A URI identifies a resource, such as `/products/P101`.
- A client retrieves or changes a resource through a uniform interface.
- The server sends a representation, such as JSON describing the product.

REST describes constraints on interactions. A list of CRUD endpoints alone does not establish those constraints.

REST & HTTP

REST is an architectural style, while HTTP is a protocol often used to implement it. HTTP provides more than a transport for JSON.

- Methods express intent, such as reading or changing a resource.
- Status codes communicate the outcome.
- Headers carry content types, cache rules, preconditions, and other metadata.
- A response body may contain JSON, XML, binary data, or another representation.

Clients should read the complete HTTP response, including its status and headers.

HTTP Request/Response As REST



15

REST Design Principle

- The web has **addressable resources**.
Each resource has a URI.
- The web has a **uniform and constrained interface**.
 - HTTP, for example, has a small number of methods. Use these to manipulate resources.
- The web is **representation oriented** – providing diverse formats.
 - Interaction with services using representations of that service.
 - Different platforms, different formats - browsers -> HTML, JavaScript -> JSON and a Java application -> XML?
- The web may be used to **communicate statelessly** – providing scalability
- **Hypermedia** is used as the **engine of application state**.
 - *Hypermedia As The Engine Of Application State (HATEOAS)*
 - Let your data formats drive state transitions in your applications.

Principle: Addressability

- every piece of data or system object is treated as a resource exposed through a unique Uniform Resource Identifier (URI).
- For example:

GET /products/P101 HTTP/1.1
Accept: application/json

the collection the product

Key distinction: the URI names *what* the client addresses; the HTTP method states *what it wants to do* with that resource.

Path Naming Rule

- Use **nouns** for resources and **plural nouns** for collections:
`/products, /orders`
- Don't use action parameter in URI, such as `/getOrders`
- Use one consistent spelling and case for fixed path segments:
`/order_items`, not a mix of `/OrderItems` and `/order_items`.
- Put the resource identifier after its collection:
`/products/{productId}`.

Path Naming Rule

- Use stable identifiers rather than names that customers or sellers may edit.

```
# Stable identity  
GET /products/P101
```

```
# Fragile if the name changes  
GET /products/travel-mug
```

Path Relations

Use nesting when the parent gives the collection a clear meaning:

```
GET /v1/customers/C7/orders
```

```
GET /v1/carts/K12/items
```

Give an independently addressable item a short, stable path where useful:

```
GET /v1/orders/055
```

Avoid requiring

`/v1/customers/C7/orders/055/items/I8/product` to reach a product. Deep paths tie clients to a long chain of relationships. Return the related product's URI or ID, or expose a simpler path.

Path Parameter vs Query Parameter

Use a **path parameter** to identify the addressed resource. Use **query parameters** to change how a collection is searched or presented.

- `/products/P101` addresses one product.
- `/products?category=mugs` addresses a filtered view of the product collection.

Note: Define allowed filters, sort keys, defaults, and maximum limits in the **API contract**.

Principle: Uniform Interface

- Clients interact with resources through a **consistent set of rules**, regardless of how the server stores them.
- HTTP Method:

Method	Safe?	Idempotent?	Typical store use
GET, HEAD	Yes	Yes	Read products or metadata
QUERY	Yes	Yes	Complex, read-only search with a body
POST	No	Not inherently	Submit checkout
PUT	No	Yes	Replace a known product representation
PATCH	No	Depends on patch semantics	Change selected product fields
DELETE	No	Yes	Remove a cart item

HTTP Method Properties

- **Safe** means a client requests an operation without intending a state change. Incidental logging does not change that intent.
- **Idempotent** means repeating an identical request has the same intended effect as sending it once. The response code need not be identical each time.

These properties help clients and intermediaries decide how to handle retries after uncertain network outcomes.

GET

- A GET requests a representation without asking to change resource state. Example:

```
GET /products/P101
```

```
GET /products?category=mugs&limit=20
```

- Cacheability depends on response headers and context; “safe” does not mean “always cached.”
- May include parameters in the URI
- **Don't use it for state changing actions, such as GET /buy?product=P101**

QUERY

RFC 10008 defines QUERY for a **safe, idempotent** request whose query input is in its body.

```
QUERY /products
```

```
Content-Type: application/json
```

```
Accept: application/json
```

```
{ "category": "mugs", "maxPriceMinor": 2000 }
```

Use it when a read-only search is too complex or lengthy for a useful URI. The target resource defines the query format and scope.

POST

- Each method call may modify the resource in a unique way
- The request may or may not contain additional information
- The response may or may not contain additional information
- The parameters are found within the request body (not within the URI)

Example:

```
POST /seller-products
```

```
Content-Type: application/json
```

```
{ "name": "Travel  
Mug", "description": "Insulated", "active": true }
```

PUT

- The PUT method **creates** a new resource or **replaces** a representation of the target resource with the request content.
- An identical replacement has the same intended effect when repeated

```
PUT /seller-products/P101
```

```
Content-Type: application/json
```

```
{"name": "Travel  
Mug", "description": "Insulated", "active": true}
```

PATCH

- The **PATCH** HTTP method applies partial modifications to a resource.
- PATCH serves as a set of instructions for modifying a resource, whereas PUT represents a complete replacement of the resource.

```
PATCH /seller-products/P101
```

```
Content-Type: application/merge-patch+json
```

```
{"name": "Larger Travel Mug"}
```

DELETE

- The **DELETE** HTTP method asks the server to delete a specified resource.
- Each method call may modify the resource in a unique way
- The request may or may not contain additional information
- The response may or may not contain additional information

DELETE /items/K12

Response codes carry meaning

Code	Meaning
200	A successful read or write with a response representation
201	A new data was created
400	Malformed or invalid request input
401 / 403	Authentication needed / authenticated caller is forbidden
404	Resource unavailable
500	Something bad happened in our server

Consistent error responses

Use HTTP Problem Details for a shared error structure when it suits the API.

```
HTTP/1.1 409 Conflict
```

```
Content-Type: application/json
```

```
{  
  "type": "https://api.example.com/problems/insufficient-  
stock",  
  "title": "Insufficient stock",  
  "status": 409,  
  "detail": "The cart requests more units than are  
available."  
}
```

Give clients a stable problem type. **Do not expose stack traces, SQL statements, or secrets.**

CRUD operation

- CRUD operations on resources performed through **HTTP methods + URI**

CRUD Operations

4 main HTTP methods

Verb

Noun

Create (Single)

POST

Collection URI

POST /products

Read (Multiple)

GET

Collection URI

GET /products

Read (Single)

GET

Entry URI

GET /products/123

Update (Single)

PUT

Entry URI

PUT /products/123

Delete (Single)

DELETE

Entry URI

DELETE /products/123

Principle: Representation Oriented (1)

- Representations of resources are exchanged.
- GET returns a representation.
- PUT and POST passes representations to the server so that underlying resources may change.
- Representations may be in many formats: XML, JSON, YAML, etc., ...

Principle: Representation Oriented (2)

- HTTP uses the CONTENT-TYPE header to specify the message format the server is sending.
- The value of the CONTENT-TYPE is a MIME typed string. Versioning information may be included.
- Examples:
 - text/plain
 - text/html
 - application/json
 - application/vnd+xml;version=1.1
- “vnd” implies a vendor specific MIME type

Principle: Representation Oriented (3)

- The ACCEPT header in content negotiation.
- An AJAX request might include a request for JSON.

Principle: Communicate Statelessly

- The application may have state but there is no client session data stored on the server.
- If there is any session-specific data it should be held and maintained by the client and transferred to the server with each request as needed.
- The server is easier to scale. No replication of session data concerns.

Principle: HATEOAS

- **Hypermedia As The Engine Of Application State**
- Hypermedia is document centric but with the additional feature of links.
- With each request returned from a server it tells you what interactions you can do next as well as where you can go to transition the state of your application.

```
<order id = "111">  
  <order-uri>http://.../order/111  
  <customer>Alice  
    <customer-uri>http://.../customers/3214  
<order-entries>  
  <order-entry>  
    <qty>5  
    <product>Nooodle  
    <product-uri>http://.../products/111
```

```
<order id = "111">  
  <customer>Alice  
  <order-entries>  
    <order-entry>  
      <qty>5  
      <product>Noodle
```



API Versioning

Backend Developer can deploy every day, but a the client may update only a few times a year. A version identifies a contract that the partner can keep using during a migration.

- A new database column or deployment number does **not** require a public API version.
- A compatible API change usually stays within the existing public version.
- A breaking change may require a parallel contract when existing clients cannot migrate together.
- Versions add routing, documentation, support, and eventual retirement work.

Design question: What will an existing client send, parse, and do after this change?

Illustration

Version 1

```
GET /v1/products/P101 HTTP/1.1
Host: api.store.example
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"id":"P101","name":"Canvas
Bag","price":25.00}
```

Version 2

```
GET /v2/products/P101 HTTP/1.1
Host: api.store.example
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"id":"P101","name":"Canvas
Bag","price":{"amountMinor":2500,"curr
ency":"USD"}}}
```

OpenAPI Specification (OAS)

OpenAPI Specification (formerly Swagger Specification) is an API description format for REST APIs. An OpenAPI file allows you to describe your entire API, including:

- Available endpoints (e.g. /users) and operations on each endpoint (e.g. GET /users, POST /users)
- Operation parameters Input and output for each operation
- Authentication methods
- Contact information, license, terms of use, and other information.

API specifications can be written in YAML or JSON. The format is easy to learn and readable to both humans and machines.

<https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.1.0.md>

Basic Structure: Metadata

Metadata

- Every API definition must include the version of the OAS

```
openapi: 3.0.0
```

Info

- The info section contains API information: title, description (optional), version, contact information, license, terms of service, and other details.

```
info:  
  title: Sample API  
  description: Optional multiline or single-line description in  
    [CommonMark](http://commonmark.org/help/) or HTML.  
  version: 0.1.9
```

Basic Structure: Servers

Servers

- The servers section specifies the API server and base URL.

```
servers:  
  - url: http://api.example.com/v1  
    description: Optional server description, e.g. Main (production) server  
  - url: http://staging-api.example.com  
    description: Optional server description, e.g. Internal staging server  
    for testing
```

Basic Structure: Paths

Paths

- The paths section defines individual endpoints (paths) in your API, and the HTTP methods (operations) supported by these endpoints. For example, GET /users can be described as:

```
paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML
      responses:
        "200":
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

OAS Complete Guide

API Server and Base Path

Media Types

Paths and Operations

Describing Parameters

Parameter Serialization

Describing Request Body >

Describing Responses

Data Models >

Adding Examples

Authentication >

Links

Callbacks

Components Section

Using \$ref

API General Info

Grouping Operations With Tags

OpenAPI Extensions



Read here:

https://swagger.io/docs/specification/v3_0/about/

REST API Design

API usability, security, and efficiency

Make it clear, sometimes depend on the Developer Experience

Clear vocabulary

Prefer names and values that communicate domain meaning. Explain unavoidable codes and identifiers.

Useful context

Return information a consumer needs to interpret the result, such as availability or currency.

Less repeated work

Consider computed data when many consumers would otherwise duplicate the same calculation.

```
# Hard to interpret
{"st": 1, "d": "20261017"}

# Clear domain vocabulary
{
  "status": "active",
  "dueDate": "2026-10-17"
}
```

Types, names, and interoperable formats

Category	Example	Contract decision
Property names	bookId, dueDate	Consistent casing and terminology
Timestamp / date	2026-09-26T08:00:00Z / 2026-10-17	Time zone and granularity
Currency / country	USD / US or USA	ISO code set and representation
Money	amountMinor: 4999, currency: USD	Units, precision and scale
Status	active	Document allowed values

Paths, query parameters, and input locations

Path parameters

Identify the target, such as `/loans/{loanId}`. OpenAPI path parameters are required.

Query parameters

Express filtering, ordering or other modifiers, such as `/books?q=web&available=true`.

Required queries can be valid

A required search term or date range can be a deliberate contract choice. Document absence and invalid values.

Bodies and headers

Bodies carry representations or change instructions. Headers carry metadata such as content type and credentials.

Filtering, sorting, and pagination

Concern	Contract example	Important decision
Filter	?category=software	Match names and define combinations
Sort	?sort=title:asc	Default order and stable tie-breaker
Offset	?offset=40&limit=20	Cost and changes between pages
Cursor	?after=token&limit=20	Opaque token and ordering semantics
Envelope	items plus next	Empty results and end-of-list signal

Actionable errors and Problem Details

Standard envelope

RFC 9457 defines application/problem+json with fields including type, title, status, detail and instance.

Useful extensions

An API can define an errors array with field pointers. Those extension fields need their own contract.

Safe feedback

Explain recovery without exposing internals. Aggregate independent validation issues when safe and useful.

HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

```
{
  "type": "urn:problem:invalid-input",
  "title": "Invalid input",
  "status": 400,
  "errors": [{
    "pointer": "/memberId",
    "detail": "Member ID is required"
  }]
}
```

Granularity, statelessness, and drafts

Useful operation boundaries

Avoid forcing every consumer through a screen-specific sequence of tiny calls. Keep business transitions clear.

Self-contained requests

Each request carries the context needed to process it. Stateless interaction still permits persistent resource state.

Saved progress

A draft application can be a resource that accepts updates before final submission and validation.

Explicit transitions

A submit operation can validate readiness and move a draft into a new business state.

API boundaries and runtime discovery

Domain boundaries

Group related capabilities coherently. API boundaries and backend deployment boundaries need not match.

Links and relations

Link headers or documented JSON links can connect resources and pagination results.

OPTIONS and Allow

A server can expose communication options and supported methods. Verify behavior rather than assuming universal discovery.

Security by design and least exposure

Authentication

Establish the caller's identity or credential context using an appropriate mechanism.

Access scope

Check token permissions when the security scheme uses scopes. Scopes constrain what an application may request.

Resource authorization

Enforce ownership, tenant, role and business rules for the specific operation and object.

Data minimization

Accept only permitted writable properties and return only data the caller may see.

Object-level authorization and BOLA (Broken Object-Level Authorization)

The risk

An authenticated caller changes an object ID and gains access to another user's data.

The required check

Validate permission for the requested object and operation, including reads, writes and list results.

Contract and tests

Document access rules and test unauthorized object IDs. OpenAPI cannot enforce ownership by itself.

```
/loans/{loanId}:  
  get:  
    summary: Read a loan  
    description: |  
      Return a loan only when the  
      caller may access it.  
      Apply ownership and tenant  
      rules in the implementation.  
      This API uses 404 to conceal  
      inaccessible loan IDs.
```

Retries and conditional updates

Problem	Mechanism	Contract behavior
Repeated creation after timeout	Documented idempotency key	Recognize one logical operation
Client reads resource	ETag: "v7"	Return an opaque representation validator
Client changes resource	If-Match: "v7"	Update only if current validator matches
Another update happened first	412 Precondition Failed	Re-read and resolve the conflict

OAuth scopes and access partitioning

Least privilege

Grant only the capabilities a client needs, using understandable permissions such as `loans:read`.

Explicit requirements

OpenAPI can define a security scheme and the scopes required by an operation.

Domain checks remain

A scope grants a class of access. It does not decide whether this caller owns this particular loan.

```
# Within securitySchemes.OAuth2
type: oauth2
flows:
  authorizationCode:
    authorizationUrl:
      https://id.example/authorize
    tokenUrl: https://id.example/token
  scopes:
    loans:read: Read permitted loans

# Operation requirement
security:
  - OAuth2: [loans:read]
```

Efficiency, quotas, and HTTP caching

Mechanism	Example	Meaning
Freshness	Cache-Control: max-age=600	May reuse while fresh for 600 seconds
Revalidation	Cache-Control: no-cache	Validate before reusing stored data
No storage	Cache-Control: no-store	Do not store this response
Conditional read	If-None-Match: "v7"	304 if unchanged, without a body
Request quota	429 with Retry-After	Follow the documented retry policy

Bulk operations and mixed outcomes

Fewer round trips

A bulk operation can reduce request overhead, but needs limits on batch size and processing cost.

Explicit execution policy

Choose atomic all-or-nothing processing or independent item outcomes. Define order and retry behavior.

A documented result format

207 originates in WebDAV. It does not standardize an arbitrary JSON batch envelope.

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"results": [
  {"requestId": "a", "status": 201,
   "location": "/loans/l1"},
  {"requestId": "b", "status": 409,
   "code": "BOOK_UNAVAILABLE"}
]}
```

```
# API-specific envelope
```

Long operations, callbacks, and events

Asynchronous work

202 Accepted means processing is not complete. A designed job resource can expose progress and the eventual result.

Polling and notifications

Clients can poll a status URI or receive a callback/webhook. Define authentication, retries and duplicate handling.

CloudEvents metadata

A standard event envelope identifies the event, source and type. Delivery guarantees remain a separate concern.

```
{  
  "specversion": "1.0",  
  "type": "org.example.loan.created",  
  "source": "/library",  
  "id": "evt-123",  
  "time": "2026-09-26T08:00:00Z",  
  "data": {"loanId": "11"}  
}
```

REST API Design

API Management

What is API management?

API management concerns publishing, controlling, protecting, and observing APIs for their consumers.

An **API gateway** is the runtime entry point that can apply shared traffic rules. Documentation, onboarding, inventory, and lifecycle processes are also part of broader API management.

Apache APISIX in the request path

- A **Route** matches an incoming request.
- An **Upstream** identifies where matching traffic goes.
- A **Consumer** represents a configured API caller for applicable policies.
- **Plugins** apply gateway behavior such as authentication, rate limits, and observability.

The Admin API or dashboard configures APISIX. Protect management access separately from public API traffic.

Example: authenticate an API consumer

APISIX's `key-auth` plugin can check an API key before forwarding a request to the store.

1. A request without the expected credential is rejected at the gateway.
2. A request with the configured credential reaches the backend.
3. The backend still checks the logged-in customer's order permissions.

An API key identifying an integration is different from an end user's store login. Define exactly which identity each layer checks.

Example: rate-limit the catalog API

The `limit-count` plugin can cap requests for a configured key and window. For an instructor demonstration, set a small limit and explicitly configure a `429` rejection response.

Benefit: one gateway policy can protect several routes from excess traffic.

Cost: a central component needs secure configuration and monitoring.

Further reading

- Microsoft Learn, [Web API Design Best Practices](#).
- Mark Massé, *REST API Design Rulebook*, Chapters 2–6 (provided reference).
- [OpenAPI 3.2 Specification](#) and [OpenAPI HTTP Methods](#).
- [RFC 10008: The HTTP QUERY Method](#)
- [Apache APISIX Documentation](#).
- Supplied class references: last year's web service slides and *Web Design: The Missing Link*.